

CUDA Graphs, Piecewise Graphs, and Compilation

Deep Dive into SGLang — Chapter 28

Yin Yang

Foundation Books Project

Where this chapter sits

This is the chapter that turns “turn on CUDA graphs” into a serving decision the runtime must prove, step by step.

- Volume I supplied the ForwardBatch: live request rows, cache locations, sequence lengths, and the output contract.
- Here the runtime reuses a recorded launch skeleton — but only after proving the live step still means the captured shape-stable state.
- One trace carries the chapter: a decode step with 37 live rows replaying a graph captured for a bucket of 40.

Goal: read a replayed step and say which rows are real, which rows are padding, and what makes the graph legal for this exact batch.

Roadmap

The replay problem

Shape-stable state

Decode replay lifecycle

Piecewise and compilation

Correctness and cost

The replay problem

Graph replay removes host launch, not GPU math

Definition (Captured graph)

A recorded execution path $G_c = (\mathcal{K}_c, \mathcal{B}_c, \sigma_c)$: kernel launches $\mathcal{K}_c$, stable buffer addresses $\mathcal{B}_c$, and a shape contract σ_c that replay must satisfy.

- Eager: the host asks the driver to enqueue *each* kernel, memcpy, and dependency — CPU setup per launch.
- Graph replay records that launch-and-dependency skeleton once and resubmits it through a cheaper host path.
- 20 tiny kernels at $\approx 8 \mu\text{s}$ launch each is $\approx 160 \mu\text{s}$ of host cost before any GPU work.

This is replay, not JIT: no PTX, cubin, or SASS is produced — the math the GPU runs is unchanged.

The price of the cheaper path is rigidity

A replayed graph remembers tensor extents, buffer addresses, stream dependencies, and output contracts. So the runtime updates *contents* inside stable buffers instead of swapping buffers.

The contract keeps **three counts** separate:

- the **live rows** the scheduler owns (37);
- the **padded bucket rows** replay sees (40);
- the **captured buffer shape** recorded at capture time.

Player-piano roll: replay changes what sits at the keys before the roll starts, but strikes the same keys in the same order.

Shape-stable state

Buckets share one graph across many live sizes

Definition (Shape bucket)

A supported padded shape β_{bucket} with padding map $\beta_b : \mathcal{R}_t \rightarrow \tilde{\mathcal{R}}_b$ that pads live rows into one replayable graph while remembering which rows are real.

For live rows $[r_1, r_2, r_3]$ and bucket 4:

$$\beta_4([r_1, r_2, r_3]) = [r_1, r_2, r_3, \perp].$$

Row $\perp$ exists only to satisfy the captured shape.

Coverage without a graph per size: nearby live sizes map to a small set of padded shapes, trading bounded padded work for more graph hits.

Fallback is a correctness path, not a second semantics

Definition (Fallback path)

The eager or separately compiled route when a live step cannot satisfy the captured shape, buffer, metadata, or output contract:

$$\text{exec}(F_t) = \begin{cases} G_c(F_t), & E_c(F_t), \\ \text{fallback}(F_t), & \text{otherwise.} \end{cases}$$

E_c is the capture-eligibility predicate. A mixed prefill, an unsupported hidden-state mode, or a token count outside the captured set takes the fallback.

The miss changes dispatch machinery, never the scheduled request.

Decode replay lifecycle

Capture front-loads work; replay just refreshes and slices

Lifecycle point	Runtime action	Why it exists
Capture	allocate stable buffers, run kernels at bucket shape, record the launch graph	stores pointer identities, not a symbolic program
Replay prepare	copy live ids, positions, seq-lens, cache locations into stable buffers; zero the tail	addresses fixed, contents refreshed
Replay	launch the retained graph once, slice back to live rows	legal only because buffers now describe this step
Fallback / recapture	eager path, or rebuild when the output contract changes	correctness wins over launch savings

A graph hit can still corrupt a step if a stale sequence length or cache location sits in a padded slot.

Piecewise and compilation

When one region is too dynamic, split the replay

Definition (Piecewise graph)

A replay plan $P = (G_1, l_1, G_2, l_2, \dots, G_m)$ of captured or compiled segments G_i separated by dynamic gaps (eager islands) l_i .

- One dynamic op or a variable token count would otherwise force the *whole* extend step back to eager.
- Piecewise keeps launch savings for the stable interior; eager cost is paid only at controlled gaps.
- A compiled segment (Inductor / backend lowering) faces the same question: does it consume and produce the live ForwardBatch?

An eager island is a controlled lifetime, output-copy, and stream-dependency boundary — not arbitrary Python in the middle of a graph.

Correctness and cost

The invariant, and the sign that decides replay

Invariant (Graph-replay step-alignment)

Captured replay may use padded tensors, but visible request order, sequence lengths, and output locations must match the scheduler decision.

Charge every step against $T_{\text{step}} = T_{\text{compute}} + T_{\text{launch}} + T_{\text{padding}} + T_{\text{metadata}}$. Replay wins only when

$$\Delta T_{\text{launch}} > \Delta T_{\text{padding}} + \Delta T_{\text{metadata}} + \Delta T_{\text{fallback}}.$$

Save 145, add 35 + 20: win by 90 μs . Add 130 + 40: lose by 25. The sign, not the graph flag, decides.

What to carry forward

1. Graph replay is a **serving decision**: prove eligibility before you reuse a recorded launch skeleton.
2. Keep **three counts** apart — live rows, padded bucket rows, captured buffer shape — and slice outputs back to live rows.
3. **Stable buffers** reuse pointer identity; their contents must be refreshed every replay, or the graph speaks for the wrong request.
4. **Piecewise** graphs and **compiled** segments extend reuse to dynamic paths through controlled eager islands.
5. A graph result is valid only when **eligibility, bucket, copy, padding, metadata, mode, and slice** name the same scheduled step.

**Next: from the replay problem to the runtime vocabulary —
captured graph, shape bucket, fallback path — that makes
one replayed step provable.**